

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\agent-af134150240932b64.jsonl`
- SHA-256: `4fad8e06b9b0bb13d6f161c0cc10fbbc5b9abaab0911b36304959308ea19cbfc`
- Source modified: `2026-06-22T18:28:25+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `subagents`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T18:24:34.581Z)

Adversarial code review of a change in the git worktree at `C:\Users\avidu\Projects\bhi-computeval` (a FastAPI badminton rally-detection engine). Find REAL bugs only ? correctness, a wrong/misleading provenance value, default-OFF/backward-compat regressions, or a broken existing consumer. No style nits. Be skeptical.

The change adds `**compute-path provenance**` (a "validation method" to prove which backend ran the DETECT). When ``/api/process`` runs, the job result now carries a ``compute`` block: ``{path: "in_process"|"cloud_run"|"not_run", requested: <picker choice>, ...}``; for the cloud path it also records the real Cloud Run ``execution`` name (verifiable via ``gcloud run jobs executions describe``). Plus ``scripts/verify_compute_path.py`` (a stdlib client that submits a job, polls, asserts the path, and cross-checks the execution in GCP).

Ground yourself by reading:

1. ``git -C C:\Users\avidu\Projects\bhi-computeval diff`` ? the full diff (backend/compute/base.py, backend/compute/cloud_run.py, backend/main.py, backend/reporting/`\_\_init\_\_`.py, tests).
2. In ``backend/main.py``: ``_execute_processing`` (the ``compute_actual`` variable, the ``finally`` block that stamps ``response["compute"]`` via ``setdefault``, and the ``[COMPUTE]`` log), and ``_maybe_dispatch_remote`` (the ``_failed`` closure's compute block + the success-path ``provenance``).
3. ``backend/compute/base.py`` (``ComputeResult.execution``) + ``backend/compute/cloud_run.py`` (``run_infer`` threading ``execution``).

Specifically probe and give a verdict on each:

- `**Provenance accuracy ? no false claims**`: can ``path`` ever say ``cloud_run`` when the job actually ran in-process, or ``in_process`` when it dispatched to cloud? Trace every return path of ``_execute_processing`` (validation fail, segmenter-not-found, segmenter Failure, in-process success, cloud success, cloud ``_failed``) and confirm the ``compute.path`` each ends up with is correct. Pay attention to the ``finally`` ``setdefault`` interplay: the cloud branch sets ``response["compute"]`` before returning (inside the ``try``), then ``finally`` runs ``setdefault`` ? does the cloud value survive, and does the in-process/fail value get stamped correctly?
- `**`compute_actual` timing**`: it's set to ``"in_process"`` only just before the local segmenter runs. For a segmenter that raises (the ``except`` re-raises after pruning) ? what ``compute.path`` does the job end up with, and is that accurate (it DID start running in-process)? Is a raise even caught here or does it propagate to the worker (and then is ``finally`` still run, stamping compute)?
- `**Default-OFF / new key**`: the response now ALWAYS has a ``compute`` key. Could any existing consumer/test break on the new key? (Search tests + the SPA contract ? the job result is returned by ``/api/jobs/{id}``.) Is a process job's behavior otherwise byte-identical?
- `**`execution` threading**`: ``run_infer`` sets ``execution=str(execution or "")``. If ``run_job`` returned None/empty, is ``execution`` an empty string (not a crash)? Does the success provenance still make sense with an empty execution?
- `**`_failed` provenance**`: for ``compute="cloud"`` but unconfigured (dispatched=False) the result says ``{path: cloud_run, requested: cloud, dispatched: False}``. Is that an accurate, non-misleading record (it did NOT run cloud)? Could a reader confuse ``path: cloud_run`` +

```
`dispatched: False` as "ran in cloud"?  
- **The report seam**: `emit_indexer_report(..., compute_path=...)` injects  
`signals["compute_path"]`. Confirm this can't raise / break a run if obsreport ignores the key  
(the report is best-effort).  
- **The verify script**: does `verify_compute_path.py` correctly FAIL (non-zero exit) on a path  
mismatch and on a cloud execution that GCP can't confirm? Any obvious bug (poll loop, exit  
codes)?
```

Return a concise findings list: severity (blocker/major/minor), exact file:line, why it's real,
concrete fix. If nothing real, say so explicitly.

2. assistant (2026-06-22T18:24:37.118Z)

I'll ground myself by reading the diff and the key files. Let me start by pulling the repo state
and reading the full diff.

3. user (2026-06-22T18:24:38.906Z)

Not currently on any branch.

Changes not staged for commit:

```
(use "git add <file>..." to update what will be committed)  
(use "git restore <file>..." to discard changes in working directory)  
    modified:   backend/compute/base.py  
    modified:   backend/compute/cloud_run.py  
    modified:   backend/main.py  
    modified:   backend/reporting/__init__.py  
    modified:   tests/test_api_cloud_dispatch.py
```

Untracked files:

```
(use "git add <file>..." to include in what will be committed)  
    scripts/verify_compute_path.py
```

no changes added to commit (use "git add" and/or "git commit -a")

---BRANCH---

---LOG---

```
8bcf6ce feat(api): make /api/compile async (202 + poll) to kill the slow-stitch 524 (#329)  
(#339)  
819e395 feat(serving): M12 ? gdrive-api:// OAuth/API StorageBackend (D14 Northstar) (#338)  
5fb55e9 fix(cloudrun): install [storage-gcs] so the cloud worker can read/write gs:// (#337)  
334ad0f feat(serving): M11 data-flywheel skeleton ? capture?shadow-eval?promote (default-OFF +  
consent faked-deny) (#336)  
6d3d054 feat(serving): SPA compute-picker ? per-request local/cloud override on /api/process  
(item 3a) (#335)
```

4. assistant (2026-06-22T18:24:41.186Z)

Let me read the full diff.

5. user (2026-06-22T18:24:42.357Z)

```
diff --git a/backend/compute/base.py b/backend/compute/base.py  
index ac4bf9a..f12fbb5 100644  
--- a/backend/compute/base.py  
+++ b/backend/compute/base.py  
@@ -41,6 +41,10 @@ class ComputeResult:  
     outputs_uri: str = ""  
     video_id: str = ""  
     report: dict = field(default_factory=dict)  
+    # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution name) ?  
+    surfaced  
+    # on the job result so "the cloud path actually ran" is independently verifiable (gcloud  
+    run jobs  
+    # executions describe <execution>). Empty for the in-process path / backends without one.  
+    execution: str = ""
```

```

class ComputeBackend(ABC):
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 86ff71e..01c2848 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -129,7 +129,8 @@ class CloudRunCompute(ComputeBackend):
    outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
    report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
    LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc)
-    return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
+    return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report,
+                          execution=str(execution or ""))

    # --- storage-backed completion signal (bucket-poll) ---
    def _read_marker(self, done_uri: str) -> Optional[dict]:
diff --git a/backend/main.py b/backend/main.py
index df08962..3c928b6 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -520,8 +520,11 @@ def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name:
str, va
    # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows (id >
play_wm);
    # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
    db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
+    # Provenance even on failure: the target was cloud_run; `dispatched` says whether the
job
+    # actually executed (False = a pre-dispatch reject ? requested cloud, never ran
remote).
    return ({ "video_id": video_id, "status": "failed", "message": msg,
-            "results": [r.model_dump() for r in val_results], "play_segments": [], ran)
+            "results": [r.model_dump() for r in val_results], "play_segments": [],
+            "compute": { "path": "cloud_run", "requested": req.compute, "dispatched":
ran}}, ran)

    # SPA compute-picker (item 3): a per-request choice overrides the server's configured
default.
    # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch (guarded by
@@ -586,9 +589,22 @@ def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name:
str, va
    # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops the
old ones.
    segments = [s for s in db_instance.query_play_segments(video_id, {}) if int(s.get("id", 0))
> play_wm]
    _finalize_reingest(video_id, segments, play_wm, cand_wm)
+    # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution` is the
real
+    # Cloud Run execution name (cross-check: gcloud run jobs executions describe <execution>).
+    provenance = {
+        "path": "cloud_run",
+        "requested": req.compute,
+        "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
+        "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
+        "execution": result.execution,
+        "outputs_uri": result.outputs_uri,
+    }
+    logger.info("[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
+               result.execution, provenance["job"], provenance["region"], result.outputs_uri,
video_id)
    return ({ "video_id": video_id, "status": "success",
            "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}')",

```

```

-         "results": [r.model_dump() for r in val_results], "play_segments": segments},
True)
+         "results": [r.model_dump() for r in val_results], "play_segments": segments,
+         "compute": provenance}, True)

def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
@@ -625,6 +641,10 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
    seg_name = req.segmenter or default_seg
    segmenter_actual = None
    segmentation_ran = False
+    # Compute-path PROVENANCE (the validation method): which backend actually ran the DETECT ?
+    # "in_process" once the local segmenter is reached, set on the response as "cloud_run" by
+    # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config fail).
+    compute_actual: Optional[str] = None
    response: Optional[Dict[str, Any]] = None
    try:
        if failures:
@@ -671,7 +691,9 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
        remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm, cand_wm,
notify)
        if remote is not None:
            response, segmentation_ran = remote # `ran` = did the cloud job actually execute
(report)
-            return response
+            return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
+            # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
+            compute_actual = "in_process"
            segmenter = segmenter_cls(db_instance, global_config)
            try:
                result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
@@ -708,13 +730,26 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
        }
        return response
    finally:
+        # Compute-path PROVENANCE stamp (the validation method): the cloud branch already set
+        # response["compute"] (path=cloud_run + the Cloud Run execution); for every other path
stamp
+        # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
+        # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is then
visible.
+        if isinstance(response, dict):
+            response.setdefault("compute", {
+                "path": compute_actual or "not_run",
+                "requested": getattr(req, "compute", None),
+            })
+        compute_path = response["compute"]["path"] if isinstance(response, dict) and "compute"
in response else None
+        logger.info("[COMPUTE] video_id=%s requested=%s actual=%s status=%s", video_id,
+            getattr(req, "compute", None), compute_path,
+            (response or {}).get("status") if isinstance(response, dict) else None)
        # Always emit the per-video observability report (next to the video, or to
report.output_dir). Never raises. Surface the paths in the response.
        paths = emit_indexer_report(
            db=db_instance, video_id=video_id, video_path=req.video_path, config=global_config,
            val_results=val_results, val_context=val_context,
            segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
-            was_reingest=was_reingest, segmentation_ran=segmentation_ran,

```

```

+         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
compute_path=compute_path,
    )
    if paths and isinstance(response, dict):
        response["reports"] = paths
diff --git a/backend/reporting/__init__.py b/backend/reporting/__init__.py
index 43c21d8..de4baf2 100644
--- a/backend/reporting/__init__.py
+++ b/backend/reporting/__init__.py
@@ -36,9 +36,13 @@ def emit_indexer_report(
    segmenter_actual: Optional[str] = None,
    was_reingest: bool = False,
    segmentation_ran: bool = True,
+    compute_path: Optional[str] = None,
) -> Optional[Dict[str, str]]:
    """Harvest + write the report. Returns the written file paths, or None if
-    reporting is unavailable/disabled/failed. Never raises."""
+    reporting is unavailable/disabled/failed. Never raises.
+
+    ``compute_path`` (the validation method) records which backend ran the DETECT ?
+    ``in_process`` / ``cloud_run`` / ``not_run`` ? so a run's report proves the compute
path."""
    config = config or {}
    # Accept either a plain dict or a typed config object (e.g. backend.config.AppConfig).
    # Normalize to a plain nested dict so harvest's dict access works uniformly.
@@ -50,6 +54,8 @@ def emit_indexer_report(
    # Consume run signals unconditionally (even when reporting is off) so the
    # process-local store never accumulates entries for finished runs.
    signals = run_signals.pop(video_id)
+    if compute_path is not None:
+        signals["compute_path"] = compute_path # flows into the report via
record_run(run_signals=?)
    if not OBSREPORT_AVAILABLE or not report_enabled(config):
        return None
    try:
diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
index 2a409ee..1d3da40 100644
--- a/tests/test_api_cloud_dispatch.py
+++ b/tests/test_api_cloud_dispatch.py
@@ -414,3 +414,99 @@ def test_cloud_override_survives_api_boundary_on_local_server(tmp_path,
monkeypatch
    segs = db.query_play_segments(job["video_id"], {})
    assert len(segs) == 2
    assert all(_meta(s)["source"] == "cloud_run" for s in segs)
+
+
+## --- compute-path PROVENANCE (the validation method: prove which backend ran)
+-----
+
+def test_cloud_dispatch_stamps_compute_provenance(cloud_env):
+    """A cloud run stamps result['compute'] with path=cloud_run + the REAL Cloud Run execution
name
+    (the GCP-verifiable proof) + the requested choice + job/region/outputs ? so the intended
path is
+    provable from the job result alone (no guessing from segment metadata)."""
+    _db, _bucket, monkeypatch = cloud_env
+    _inject_fake(monkeypatch, _FakeCloudClient())
+
+    out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid",
+
+                                     compute="cloud"))
+
+    prov = out["compute"]
+    assert prov["path"] == "cloud_run"
+    assert prov["requested"] == "cloud"
+    assert prov["execution"] == "exec-1" # the fake client's execution name (real one in

```

```

prod)
+   assert prov["outputs_uri"].endswith("/outputs/vidCloud/")
+   assert prov["job"] and prov["region"]           # the dispatch coordinates are recorded
+
+
+def test_local_override_stamps_in_process_provenance(cloud_env):
+   """On a cloud server, compute='local' runs in-process AND the result proves it:
path=in_process,
+   requested=local ? so a 'ran local despite a cloud server' is auditable (no silent
ambiguity)."""
+   _db, _bucket, monkeypatch = cloud_env
+   _inject_fake(monkeypatch, _FakeCloudClient())
+   monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
+
+   out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="motion",
+
+                                           compute="local"))
+   assert out["status"] == "success"
+   assert out["compute"] == {"path": "in_process", "requested": "local"}
+
+
+def test_default_in_process_stamps_provenance(tmp_path, monkeypatch):
+   """A default-OFF (no cloud) run stamps path=in_process, requested=None ? the byte-identical
local
+   path is now self-describing."""
+   db = Database(str(tmp_path / "t.db"))
+   monkeypatch.setattr(m, "db_instance", db)
+   monkeypatch.setattr(m, "global_config", AppConfig())
+   monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
+   monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
+   video = tmp_path / "v.mp4"
+   video.write_bytes(b"hashable-bytes")
+
+   out = m._execute_processing(m.ProcessRequest(video_path=str(video), segmenter="motion"))
+   assert out["compute"] == {"path": "in_process", "requested": None}
+
+
+def test_cloud_requested_unconfigured_stamps_not_dispatched(tmp_path, monkeypatch):
+   """compute='cloud' on a server that can't dispatch fails clearly AND records it: target
cloud_run,
+   requested=cloud, dispatched=False ? proving the cloud job never ran (no silent local
fallback)."""
+   db = Database(str(tmp_path / "t.db"))
+   monkeypatch.setattr(m, "db_instance", db)
+   monkeypatch.setattr(m, "global_config", AppConfig.model_validate({"deployment":
{"compute_target": ""}}))
+   monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
+   monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
+   monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
+   video = tmp_path / "v.mp4"
+   video.write_bytes(b"hashable-bytes")
+
+   out = m._execute_processing(m.ProcessRequest(video_path=str(video),
segmenter="wasb_hybrid",
+
+                                           compute="cloud"))
+   assert out["status"] == "failed"
+   assert out["compute"] == {"path": "cloud_run", "requested": "cloud", "dispatched": False}
+
+
+def test_validation_failure_stamps_not_run_provenance(tmp_path, monkeypatch):
+   """A validation failure never reaches a segmenter ? path=not_run (nothing executed)."""
+   db = Database(str(tmp_path / "t.db"))
+   monkeypatch.setattr(m, "db_instance", db)
+   monkeypatch.setattr(m, "global_config", AppConfig())
+   fail = [ValidationResult(validator_name="x", passed=False, message="bad", details={})]

```

```

+ monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (fail, {}))
+ video = tmp_path / "v.mp4"
+ video.write_bytes(b"hashable-bytes")
+
+ out = m._execute_processing(m.ProcessRequest(video_path=str(video)))
+ assert out["status"] == "failed"
+ assert out["compute"]["path"] == "not_run"
+
+
+def test_full_api_process_job_result_carries_cloud_provenance(cloud_env):
+    """End-to-end: POST /api/process (compute=cloud) ? the job result the SPA polls carries the
+    cloud_run provenance + execution, so the UI/operator can PROVE the cloud path was taken."""
+    _db, _bucket, monkeypatch = cloud_env
+    _inject_fake(monkeypatch, _FakeCloudClient(video_id="vidProv"))
+    client = TestClient(m.app)
+
+    r = client.post("/api/process",
+                    json={"video_path": "gs://b/clip.mp4", "segmenter": "wasb_hybrid",
+                        "compute": "cloud"})
+    assert r.status_code == 202
+    job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
+    assert job["status"] == "done"
+    assert job["result"]["compute"]["path"] == "cloud_run"
+    assert job["result"]["compute"]["execution"] == "exec-1"

```

6. assistant (2026-06-22T18:24:46.062Z)

Now let me read the key sections of `backend/main.py` in detail, especially `\_execute\_processing` and `\_maybe\_dispatch\_remote`.

7. user (2026-06-22T18:24:46.818Z)

```

480             try:
481                 meta["shot_count"] = int(float(sc)) # OverflowError on
'inf'/'1e400'
482             except (ValueError, OverflowError):
483                 meta["shot_count"] = sc
484                 db_instance.add_play_segment(video_id, start, end, metadata=meta)
485                 n += 1
486         return n
487
488
489     def _cloud_available() -> bool:
490         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
491         3).
492         True when the control plane is wired for Cloud Run: either the configured default
493         target is
494         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
495         coordinates
496         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
497         (``storage.output_root``)
498         are present so a forced per-request override can actually dispatch + collect a
499         result.
500         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
501         would work."""
502         deployment = getattr(global_config, "deployment", None)
503         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
504             return True
505         has_job = bool(os.environ.get("CLOUD_RUN_JOB") and
os.environ.get("GOOGLE_CLOUD_PROJECT"))
506         out_root = (getattr(getattr(global_config, "storage", None), "output_root", "") or
"").strip()
507         return bool(has_job and out_root)

```

```

504
505
506     def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str,
val_results,
507                                     play_wm: int, cand_wm: int, notify) ->
Optional[Tuple[Dict[str, Any], bool]]:
508         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
`deployment.compute_target
509         == "cloud_run"`, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
510         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
511
512         Returns `(response_dict, ran)` when the cloud path is taken ? `ran` is whether
the cloud job
513         actually EXECUTED (`False` for a pre-dispatch config rejection, so the
observability report
514         doesn't claim a fake segmentation run) ? or `None` to fall through to the in-
process segmenter
515         (**DEFAULT-OFF*: `get_compute_backend` returns `None` for every
non-`cloud_run` target, so
516         the in-process path stays byte-identical)."""
517         from backend.compute import ComputeJobSpec, get_compute_backend
518
519         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
520             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
521             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
522             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
523             # Provenance even on failure: the target was cloud_run; `dispatched` says
whether the job
524             # actually executed (False = a pre-dispatch reject ? requested cloud, never ran
remote).
525             return ({ "video_id": video_id, "status": "failed", "message": msg,
526                     "results": [r.model_dump() for r in val_results], "play_segments": [],
527                     "compute": { "path": "cloud_run", "requested": req.compute,
"dispatched": ran}}, ran)
528
529         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
530         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
531         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
532         choice = (req.compute or "").strip().lower()
533         if choice and choice not in ("local", "cloud"):
534             logger.warning("[API] ignoring unknown compute=%r (expected 'local'/'cloud');
using server default.",
535                             req.compute)
536         choice = ""
537         if choice == "local":
538             return None # explicit "run on my machine" ? in-process regardless of the
server's target
539         if choice == "cloud":
540             if not _cloud_available():
541                 return _failed("cloud-serving was requested but this server isn't configured
for it "
542                               "(no Cloud Run target). Pick 'run on my machine', or
configure cloud serving.",
543                               False)
544             compute = get_compute_backend(global_config, target_override="cloud_run")
545         else:
546             compute = get_compute_backend(global_config) # server default (default-OFF
unless cloud_run)
547         if compute is None:
548             return None # default-OFF ? run the in-process segmenter below (unchanged)

```

```

549
550     storage_cfg = getattr(global_config, "storage", None)
551
552     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
553     try:
554         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
555     except ValueError as e:
556         return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}",
False)
557     if input_ref.backend == "local":
558         return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to
this machine "
559                        "can't be read by the Cloud Run job. Put the video in your bucket
first.", False)
560     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
561     if not out_root:
562         return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
563                        "? that's where the Cloud Run job writes the result.", False)
564
565     notify("dispatching (cloud_run)")
566     logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
567     spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root,
segmenter=seg_name)
568     try:
569         result = compute.run_infer(spec) # dispatch the Cloud Run Job + bucket-poll to
completion
570     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
571         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
572         return _failed(f"cloud dispatch error: {e}", True)
573     if result.returncode != 0:
574         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
575     if result.video_id and result.video_id != video_id:
576         # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
577         # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
578         logger.warning("[API] cloud video_id %s != local %s ? input bytes differ between
the API hash "
579                        "and the worker hash; ingesting under the local id.",
result.video_id, video_id)
580
581     notify("ingesting (cloud_run)")
582     storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else
{}
583     try:
584         _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
585     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
586         logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
587         return _failed(f"cloud-serving: could not read the result from
{result.outputs_uri}: {e}", True)
588
589     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
590     segments = [s for s in db_instance.query_play_segments(video_id, {})] if
int(s.get("id", 0)) > play_wm]
591     _finalize_reingest(video_id, segments, play_wm, cand_wm)
592     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
593     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).

```

```

594     provenance = {
595         "path": "cloud_run",
596         "requested": req.compute,
597         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
598         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
599         "execution": result.execution,
600         "outputs_uri": result.outputs_uri,
601     }
602     logger.info("[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
603               result.execution, provenance["job"], provenance["region"],
result.outputs_uri, video_id)
604     return ({ "video_id": video_id, "status": "success",
605             "message": f"Successfully indexed {len(segments)} play segments (cloud_run
/ '{seg_name}')." ,
606             "results": [r.model_dump() for r in val_results], "play_segments":
segments,
607             "compute": provenance}, True)
608
609
610 def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
611     """The full hash ? validate ? segment ? report pipeline for one video.
612
613     This is the former synchronous /api/process body, unchanged in behavior, now run on
614     the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
615     the sync endpoint used to return (UI compatibility); the per-video observability
616     report is still emitted in `finally:` and grafted as response["reports"].
617
618     `progress` is an optional callable(stage: str) used to surface coarse job progress.
619     Raises on unexpected internal errors ? the caller records those as a job failure
620     (after this function has already restored the pre-run segments, see below).
621     """
622     notify = progress or (lambda stage: None)
623
624     notify("hashing")
625     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
626     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
627     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
628     # consumers (hash / validators / segmenter) use the resolved local path.
629     local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
630     video_id = compute_video_id(local_video)
631     was_reingest = db_instance.get_video(video_id) is not None
632
633     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
634     notify("validating")
635     logger.info(f"Running validations for {req.video_path}...")
636     val_results, val_context = registry.run_all_with_context(local_video, global_config)
637     failures = [r for r in val_results if not r.passed]
638
639     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
640     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
641     seg_name = req.segmenter or default_seg
642     segmenter_actual = None
643     segmentation_ran = False
644     # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
645     # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
646     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config

```

```

fail).
647     compute_actual: Optional[str] = None
648     response: Optional[Dict[str, Any]] = None
649     try:
650         if failures:
651             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
652             # status; it no longer destroys the video's prior good segments.
653             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
654             response = {
655                 "video_id": video_id,
656                 "status": "failed",
657                 "message": "Video failed validation checks.",
658                 "results": [r.model_dump() for r in val_results],
659             }
660             return response
661
662         # 2. Run segmenter & indexer
663         logger.info("[API] Video passed checks. Segmenting and indexing...")
664         db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
665
666         segmenter_cls = segmenter_registry.get(seg_name)
667         if not segmenter_cls:
668             # Submit-time validation already 400s unknown names; this is the defensive
669             # in-worker path (e.g. config default pointing at an unregistered
segmenter).
670             available = list(segmenter_registry.list_available().keys())
671             response = {
672                 "video_id": video_id,
673                 "status": "failed",
674                 "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
675                 "results": [r.model_dump() for r in val_results],
676                 "play_segments": [],
677             }
678             return response
679
680         logger.info(f"[API] Using segmenter: {seg_name}")
681         notify(f"segmenting ({seg_name})")
682         segmenter_actual = seg_name
683         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
684         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
685         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
686         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
687         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
688         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
689         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
690         # they're dropped on the cloud path in this slice ? a documented follow-up.)
691         remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm,
cand_wm, notify)
692         if remote is not None:
693             response, segmentation_ran = remote # `ran` = did the cloud job actually
execute (report)
694             return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
695         # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
696         compute_actual = "in_process"

```

```

697         segmenter = segmenter_cls(db_instance, global_config)
698     try:
699         result = segmenter.process_video(video_id, local_video, req.player_pool,
700 req.max_frames)
701     except Exception:
702         # A raising segmenter must not leave half-written rows: restore the pre-run
703         # state (same destroy-after-confirm contract as the Failure branch), then
704         let
705             # the job worker record the failure.
706             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
707             raise
708     segmentation_ran = True
709
710     if isinstance(result, Failure):
711         logger.error(f"[API] Segmenter failed: {result.message}")
712         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
713         response = {
714             "video_id": video_id,
715             "status": "failed",
716             "message": f"Segmenter '{seg_name}' failed: {result.message}",
717             "results": [r.model_dump() for r in val_results],
718             "play_segments": [],
719         }
720         return response
721
722     segments = result.value if hasattr(result, "value") else result
723     notify("finalizing")
724     _finalize_reingest(video_id, segments, play_wm, cand_wm)
725
726     response = {
727         "video_id": video_id,
728         "status": "success",
729         "message": f"Successfully indexed {len(segments)} play segments using
730 '{seg_name}' segmenter.",
731         "results": [r.model_dump() for r in val_results],
732         "play_segments": segments,
733     }
734     return response
735
736 finally:
737     # Compute-path PROVENANCE stamp (the validation method): the cloud branch
738     already set
739     # response["compute"] (path=cloud_run + the Cloud Run execution); for every
740     other path stamp
741     # the actual path here so the job result PROVES which backend ran (in_process /
742     not_run) and
743     # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
744     then visible.
745     if isinstance(response, dict):
746         response.setdefault("compute", {
747             "path": compute_actual or "not_run",
748             "requested": getattr(req, "compute", None),
749         })
750     compute_path = response["compute"]["path"] if isinstance(response, dict) and
751     "compute" in response else None
752     logger.info("[COMPUTE] video_id=%s requested=%s actual=%s status=%s", video_id,
753         getattr(req, "compute", None), compute_path,
754         (response or {}).get("status") if isinstance(response, dict) else
755         None)
756     # Always emit the per-video observability report (next to the video, or to
757     # report.output_dir). Never raises. Surface the paths in the response.
758     paths = emit_indexer_report(
759         db=db_instance, video_id=video_id, video_path=req.video_path,
760         config=global_config,
761         val_results=val_results, val_context=val_context,
762         segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,

```

```

752         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
compute_path=compute_path,
753     )
754     if paths and isinstance(response, dict):
755         response["reports"] = paths
756
757
758     @app.post("/api/process", status_code=202)
759     def process_video(req: ProcessRequest, request: Request):

```

8. assistant (2026-06-22T18:24:49.319Z)

Now let me look at the cloud_run.py `run\_infer` method to understand the `execution` threading and confirm what happens when `run\_job` returns None/empty.

9. user (2026-06-22T18:24:50.555Z)

```

1     """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2     for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4     Flow (the cloud-serving compute path):
5     1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6         --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7     2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8         completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9         it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10        cold-start is absorbed by the async poll.
11    3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12        ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14    The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17    The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18    with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19    owner-verified on the M7 real run.
20    """
21
22    from __future__ import annotations
23
24    import json
25    import logging
26    import os
27    import tempfile
28    import uuid
29    from abc import ABC, abstractmethod
30    from typing import Any, List, Optional
31
32    from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
34    from backend.storage import fetch, parse_uri
35    from backend.storage import get_storage
36
37    LOG = logging.getLogger("compute.cloud_run")

```

```

38
39     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
40     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
41     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
42         "deployment.compute_target=local_gpu",
43         "indexing.detector_impl=native",
44     )
45
46
47     class CloudRunJobClient(ABC):
48         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
49
50         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
51         overrides the job's container ``args`` for this execution and starts it."""
52
53         @abstractmethod
54         def run_job(self, job_name: str, argv: List[str], *, region: str,
55                     project: Optional[str] = None) -> str:
56             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
57             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
58
59
60         class GoogleCloudRunJobClient(CloudRunJobClient):
61             """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
62             a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
63
64             def run_job(self, job_name: str, argv: List[str], *, region: str,
65                         project: Optional[str] = None) -> str:
66                 try:
67                     from google.cloud import run_v2 # type: ignore[attr-defined]
68                 except Exception as exc: # pragma: no cover - real SDK not present in CI
69                     raise RuntimeError(
70                         "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                         "control plane (it is intentionally NOT a CI/test dependency).")
72                 ) from exc
73                 client = run_v2.JobsClient() # pragma: no cover - exercised only on the real M7
run
74                 name = client.job_path(project, region, job_name)
75                 overrides = run_v2.RunJobRequest.Overrides(
76                     container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77                 )
78                 op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79                 return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
80
81
82         class CloudRunCompute(ComputeBackend):
83             """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
84
85             def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
86                         project: Optional[str] = None, storage_config: Optional[dict] = None,
87                         policy: ExecutionPolicy = DEFAULT_POLICY,
88                         token: Optional[str] = None,
89                         container_overrides: Optional[tuple[str, ...]] = None) -> None:

```

```

90         self._client = run_client
91         self._job_name = job_name
92         self._region = region
93         self._project = project
94         self._storage_config = storage_config or {}
95         self._policy = policy
96         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98         self._token = token
99         self._container_overrides = (
100             _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101         )
102
103     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104         out_root = spec.out_uri.rstrip("/")
105         token = self._token or uuid.uuid4().hex
106         done_uri = f"{out_root}/_dispatch/{token}.done"
107         argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
108                             "--done-uri", done_uri]
109         if spec.segmenter:
110             argv += ["--segmenter", spec.segmenter]
111         for kv in (*spec.config_overrides, *self._container_overrides):
112             argv += ["--set", kv]
113
114         execution = self._client.run_job(self._job_name, argv, region=self._region,
project=self._project)
115         LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
self._job_name, execution, done_uri)
116
117         marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
label="cloud-run-infer", logger=LOG)
118         video_id = str(marker.get("video_id", ""))
119         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
120         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
121         rc = int(marker.get("returncode", 1))
122         if not video_id:
123             LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
124                         "video_id) ? treating as failure", self._job_name)
125             rc = rc or 1
126         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
127         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
128         LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
129         return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report,
130                             execution=str(execution or ""))
131
132     # --- storage-backed completion signal (bucket-poll) ---
133
134
135

```

10. assistant (2026-06-22T18:24:54.386Z)

Now let me read the `ComputeResult` definition in base.py and verify the report seam in reporting/__init__.py.

11. user (2026-06-22T18:24:55.619Z)

```

20     @dataclass(frozen=True)
21     class ComputeJobSpec:

```

```

22         """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23         from the SAME input bytes, so the output always lands at
24         ``<out_uri>/outputs/<video_id>/`` ?
25         deterministic regardless of where the job ran."""
26         video_uri: str                                # input (gs://? for cloud; a path/local://
for local)
27         out_uri: str                                  # output ROOT (gs://? bucket for cloud)
28         segmenter: Optional[str] = None
29         # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30         config_overrides: tuple[str, ...] = ()
31
32
33     @dataclass(frozen=True)
34     class ComputeResult:
35         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36         (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
37         empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
38         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
39
40         returncode: int
41         outputs_uri: str = ""
42         video_id: str = ""
43         report: dict = field(default_factory=dict)
44         # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution
name) ? surfaced
45         # on the job result so "the cloud path actually ran" is independently verifiable
(gcloud run jobs
46         # executions describe <execution>). Empty for the in-process path / backends without
one.
47         execution: str = ""
48
49
50     class ComputeBackend(ABC):
51         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
52         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
53         execution identically."""
54
55         @abstractmethod
56         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
57             """Run ``spec`` to completion and return its ``ComputeResult``."""
58

```

12. assistant (2026-06-22T18:24:58.980Z)

Now let me read the reporting `\_\_init\_\_.py` to verify the report seam and check how `record\_run` / `run\_signals` flows, to confirm the `compute\_path` key can't break obsreport.

13. user (2026-06-22T18:24:59.795Z)

```

1         """Per-video observability reports for the indexer.
2
3         `obsreport` is a SOFT dependency: if it is not installed, this module degrades to a
4         no-op and the pipeline runs exactly as before. The single entrypoint
5         `emit_indexer_report(...)` is designed to be called from a `finally` block and
6         NEVER raises ? emitting a report must not be able to break a processing run.
7         """

```

```

8     from __future__ import annotations
9
10    from typing import Any, Dict, List, Optional
11
12    from . import run_signals # no obsreport dependency ? always importable
13
14    try: # soft dependency ? never hard-fail the pipeline on a reporting import
15        import obsreport # noqa: F401
16        from . import harvest
17
18        OBSREPORT_AVAILABLE = True
19    except Exception: # pragma: no cover - exercised only when obsreport is absent
20        OBSREPORT_AVAILABLE = False
21
22
23    def report_enabled(config: Optional[Dict[str, Any]]) -> bool:
24        return bool((config or {}).get("report", {}).get("enabled", True))
25
26
27    def emit_indexer_report(
28        *,
29        db,
30        video_id: str,
31        video_path: Optional[str],
32        config: Optional[Dict[str, Any]],
33        val_results: List[Any],
34        val_context: Optional[Dict[str, Any]] = None,
35        segmenter_requested: Optional[str] = None,
36        segmenter_actual: Optional[str] = None,
37        was_reingest: bool = False,
38        segmentation_ran: bool = True,
39        compute_path: Optional[str] = None,
40    ) -> Optional[Dict[str, str]]:
41        """Harvest + write the report. Returns the written file paths, or None if
42        reporting is unavailable/disabled/failed. Never raises.
43
44        ``compute_path`` (the validation method) records which backend ran the DETECT ?
45        ``in_process`` / ``cloud_run`` / ``not_run`` ? so a run's report proves the compute
46        path."""
47        config = config or {}
48        # Accept either a plain dict or a typed config object (e.g.
49        backend.config.AppConfig).
50        # Normalize to a plain nested dict so harvest's dict access works uniformly.
51        if hasattr(config, "model_dump"):
52            try:
53                config = config.model_dump(mode="json")
54            except Exception:
55                config = {}
56        # Consume run signals unconditionally (even when reporting is off) so the
57        # process-local store never accumulates entries for finished runs.
58        signals = run_signals.pop(video_id)
59        if compute_path is not None:
60            signals["compute_path"] = compute_path # flows into the report via
61            record_run(run_signals=?)
62        if not OBSREPORT_AVAILABLE or not report_enabled(config):
63            return None
64        try:
65            play_segments = db.query_play_segments(video_id, {}) if db is not None else []
66            candidates = db.query_candidate_segments(video_id) if db is not None else []
67            rec = harvest.record_run(
68                video_id=video_id,
69                video_path=video_path,
70                config=config,
71                val_results=val_results,
72                val_context=val_context,

```

```

70         play_segments=play_segments,
71         candidates=candidates,
72         segmenter_requested=segmenter_requested,
73         segmenter_actual=segmenter_actual,
74         was_reingest=was_reingest,
75         segmentation_ran=segmentation_ran,
76         run_signals=signals,
77     )
78     output_dir = (config.get("report") or {}).get("output_dir")
79     res = rec.write(output_dir=output_dir)
80     return res.paths or None
81 except Exception: # pragma: no cover - defensive; reporting must never break a run
82     return None
83

```

14. assistant (2026-06-22T18:25:05.145Z)

The report seam is safe: `compute\_path` only writes into the `signals` dict, the entire `try` is wrapped in `except Exception: return None`, and the function never raises. Let me check `run\_signals.pop` to confirm it returns a fresh dict even when no signals were recorded (so the `signals["compute\_path"]` assignment can't fail).

15. user (2026-06-22T18:25:05.607Z)

```

1     """In-process signal channel: pipeline internals -> the per-video report.
2
3     Some report-worthy facts (e.g. the provider refusing an oversize chunk) surface
4     deep inside a segmenter's worker threads, with no return path to the harvest ?
5     segmenters return only their segment list. Layers record those facts here keyed
6     by video_id; `emit_indexer_report` pops them when the run's report is built.
7
8     Process-local and best-effort BY DESIGN: recording must never break a run, the
9     store is consumed (popped) once per report, and signals for a video are simply
10    dropped if no report is emitted. No obsreport dependency ? importable everywhere.
11    """
12    from __future__ import annotations
13
14    import threading
15    from typing import Any, Dict
16
17    _lock = threading.Lock()
18    _signals: Dict[str, Dict[str, Any]] = {}
19
20
21    def incr(video_id: str, key: str, by: int = 1) -> None:
22        """Add `by` to the counter `key` for this video's in-flight run."""
23        if not video_id or by == 0:
24            return
25        with _lock:
26            sig = _signals.setdefault(video_id, {})
27            sig[key] = int(sig.get(key, 0)) + by
28
29
30    def pop(video_id: str) -> Dict[str, Any]:
31        """Consume (remove and return) all signals recorded for this video's run."""
32        with _lock:
33            return _signals.pop(video_id, {})
34

```

16. assistant (2026-06-22T18:25:09.108Z)

`pop` always returns a dict (default `{}`), so `signals["compute\_path"] = compute\_path` is safe. Now let me verify the verify script. Let me read it in full.

17. user (2026-06-22T18:25:09.583Z)

```
1      #!/usr/bin/env python3
2      """Verify the compute path an /api/process job ACTUALLY took (COMPUTE_DECOUPLED
validation method).
3
4      Submit a video with a chosen compute target, poll the job, and ASSERT
5      ``result["compute"]["path"]`` matches the intent ? then, for a cloud run, independently
cross-check
6      that the recorded Cloud Run **execution** really exists in GCP (`gcloud run jobs
executions describe`).
7      This closes the loop: the picker asked for a path, and we prove the engine took it (no
silent fallback).
8
9      Exit 0 = the intended path was provably taken; non-zero = a mismatch / the cloud
execution couldn't
10     be confirmed. Stdlib-only (urllib) so it runs anywhere; `--verify-gcp` shells out to
`gcloud`.
11
12     Examples:
13     # local engine, cloud path, with the GCP cross-check
14     python scripts/verify_compute_path.py --base http://127.0.0.1:8000 \
15         --video gs://khelsutra-rally-corpus/videos/mahadevpura_smoke_180s.mp4 \
16         --segmenter wasb_hybrid --compute cloud --expect cloud_run --verify-gcp \
17         --region asia-southeast1 --project gen-lang-client-0306026826
18
19     # prove a local request stays in-process
20     python scripts/verify_compute_path.py --base http://127.0.0.1:8000 \
21         --video gs://b/clip.mp4 --compute local --expect in_process
22     """
23     import argparse
24     import json
25     import subprocess
26     import sys
27     import time
28     import urllib.request
29
30
31     def _post(base: str, path: str, body: dict):
32         req = urllib.request.Request(base.rstrip("/") + path,
data=json.dumps(body).encode(),
33                                     headers={"Content-Type": "application/json"},
method="POST")
34         with urllib.request.urlopen(req, timeout=30) as r: # noqa: S310 ? operator-supplied
base URL
35             return r.status, json.loads(r.read().decode())
36
37
38     def _get(base: str, path: str):
39         with urllib.request.urlopen(base.rstrip("/") + path, timeout=30) as r: # noqa: S310
40             return r.status, json.loads(r.read().decode())
41
42
43     def main() -> int:
44         ap = argparse.ArgumentParser(description="Prove which compute path an /api/process
job took.")
45         ap.add_argument("--base", required=True, help="engine base URL, e.g.
http://127.0.0.1:8000")
46         ap.add_argument("--video", required=True, help="video_path (a gs:// URI for the
cloud path)")
47         ap.add_argument("--segmenter", default=None)
48         ap.add_argument("--compute", choices=["local", "cloud"], default=None,
49                         help="the picker choice to send (omit ? server default)")
50         ap.add_argument("--expect", choices=["in_process", "cloud_run"], required=True,
51                         help="the path the job MUST report taking")
```

```

52     ap.add_argument("--region", default="asia-southeast1", help="fallback Cloud Run
region for --verify-gcp")
53     ap.add_argument("--project", default=None, help="GCP project for --verify-gcp")
54     ap.add_argument("--verify-gcp", action="store_true",
55                     help="cross-check the recorded Cloud Run execution exists in GCP
(gcloud)")
56     ap.add_argument("--timeout-sec", type=float, default=1200)
57     args = ap.parse_args()
58
59     body: dict = {"video_path": args.video}
60     if args.segmenter:
61         body["segmenter"] = args.segmenter
62     if args.compute:
63         body["compute"] = args.compute
64
65     status, sub = _post(args.base, "/api/process", body)
66     if status != 202:
67         print(f"FAIL submit: HTTP {status}: {sub}")
68         return 2
69     job_id = sub["job_id"]
70     print(f"submitted job {job_id}; polling /api/jobs/{job_id} ?")
71
72     started = time.monotonic()
73     st, job = "queued", {}
74     while st not in ("done", "failed"):
75         if time.monotonic() - started > args.timeout_sec:
76             print("FAIL: job poll timed out")
77             return 3
78         time.sleep(5)
79         _, job = _get(args.base, f"/api/jobs/{job_id}")
80         st = job.get("status", "queued")
81         print(f" {st} ? {job.get('progress')}")
82
83     result = job.get("result") or {}
84     prov = result.get("compute") or {}
85     print(f"job {st}; result.status={result.get('status')}; compute={json.dumps(prov)}")
86
87     actual = prov.get("path")
88     if actual != args.expect:
89         print(f"FAIL: intended path {args.expect!r} but the job took {actual!r}")
90         return 4
91     print(f"OK: the job took the intended compute path: {actual}")
92
93     if args.expect == "cloud_run" and args.verify_gcp:
94         execu = prov.get("execution")
95         if not execu:
96             print("FAIL: cloud_run path but no execution name recorded ? cannot cross-
check")
97             return 5
98         cmd = ["gcloud", "run", "jobs", "executions", "describe", execu,
99               "--region", prov.get("region") or args.region,
100               "--format",
101               "value(metadata.name,status.succeededCount,status.failedCount)"]
102         if args.project:
103             cmd += ["--project", args.project]
104         print("cross-checking in GCP:", " ".join(cmd))
105         cp = subprocess.run(cmd, capture_output=True, text=True) # noqa: S603
106         if cp.returncode != 0:
107             print(f"FAIL: GCP could not confirm execution {execu}: {cp.stderr.strip()}")
108             return 6
109         print(f"OK: GCP confirms the Cloud Run execution {execu}: {cp.stdout.strip()}")
110
111     print("VERIFIED ? the intended compute path was provably taken.")
112     return 0

```

```

113
114     if __name__ == "__main__":
115         sys.exit(main())
116

```

18. assistant (2026-06-22T18:25:14.610Z)

Now I have a thorough understanding. Let me trace a critical edge case about the `\_failed` dispatched flag. There's a subtle issue: in `\_failed`, when `dispatched=True` but the job actually failed (e.g. returncode != 0, ingest error), the result says `{path: cloud\_run, dispatched: True}`. Let me check the worker-side path to confirm whether a raise from the segmenter still runs `finally`. Let me check the worker invocation of `\_execute\_processing`.

19. user (2026-06-22T18:25:16.081Z)

```

C:\Users\avidu\Projects\bhi-computeval\backend\main.py:610:def _execute_processing(req:
ProcessRequest, progress=None) -> Dict[str, Any]:
C:\Users\avidu\Projects\bhi-computeval\backend\api\job_worker.py:9:every cross-function seam ?
`db_instance`, `global_config`, `_execute_processing`,
C:\Users\avidu\Projects\bhi-computeval\backend\api\job_worker.py:46:         is the segmenter's
`_execute_processing`, run UNDER the destroy-after-confirm contract.
C:\Users\avidu\Projects\bhi-computeval\backend\api\job_worker.py:69:         # v8 compute-picker:
the worker runs _execute_processing ? _maybe_dispatch_remote, which
C:\Users\avidu\Projects\bhi-computeval\backend\api\job_worker.py:98:         result =
_main._execute_processing(

```

20. user (2026-06-22T18:25:18.715Z)

```

80         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
81         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
82         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
83         job self-scheduled and will be re-claimed when `next_poll_at` is due.
84         """
85         import backend.main as _main
86         db_instance = _main.db_instance
87         job_id = job["id"]
88         # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
89         # original /api/process pipeline). Both record their terminal state the same way;
only process
90         # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
91         is_compile = job.get("kind") == "compile"
92         try:
93             if is_compile:
94                 result = _main._execute_compile(
95                     job, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
96             else:
97                 req = _main._job_to_request(job)
98                 result = _main._execute_processing(
99                     req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
100             if result.get("video_id"):
101                 db_instance.set_job_video_id(job_id, result["video_id"])
102             db_instance.mark_job_done(job_id, result)
103             if not is_compile:
104                 _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF; process
jobs only)
105                 _maybe_run_flywheel(job, result) # M11 data-flywheel (process jobs only)
106         except _main.JobWaiting as w:
107             # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
108             logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
109             db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)

```

```

110         except Exception as e:
111             logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
112             db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
113
114
115     def _maybe_record_job_cost(job: Dict[str, Any], result: Dict[str, Any]) -> None:
116         """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
117         ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
118         (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
119         pricebook, and
120         persist it.
121
122         ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
123         so even with
124         ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
125         **0** until a
126         FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
127         ledger + pricebook
128         + the recording seam; the usage-emission step is separate (the metering hooks on the
129         real run).
130         With the placeholder pricebook the cost is 0 regardless.
131
132         Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
133         is the
134         claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
135         import backend.main as _main
136
137         billing_cfg = getattr(_main.global_config, "billing", None)
138         if not billing_cfg or not getattr(billing_cfg, "enabled", False):
139             return
140         try:
141             usage = (result or {}).get("usage") or {}
142             _main.db_instance.record_job_cost(
143                 job["id"], usage=usage, pricebook_version=billing_cfg.pricebook_version,
144                 gpu_type=usage.get("gpu_type"), compute_backend=(result or
145                 {}).get("compute_backend"),
146                 owner_id=job.get("owner_id"), margin=billing_cfg.margin,

```

21. assistant (2026-06-22T18:25:24.988Z)

Now I understand the critical worker behavior. When `_execute_processing`` raises (e.g. segmenter raises), the worker catches it at line 110 and calls `mark_job_failed`` ? but crucially, when the segmenter raises inside `_execute_processing``, the `finally`` block `**does run**` (re-raise from the inner `except`` still triggers the outer `finally``), but at that point `response`` is still `None`` (the `except`` re-raises before any `response = ...`` is set). So `setdefault`` on `None`` is skipped (the `isinstance(response, dict)`` guard fails). This means a segmenter-raise produces NO `compute`` block in the result ? and in fact the worker marks the job `failed`` with just an error string (no `result`` dict at all).

Let me verify what `mark_job_failed`` stores and whether `/api/jobs/{id}`` returns a `result`` in that case. Let me also confirm the SPA contract / consumers of the `compute`` key.

22. user (2026-06-22T18:25:26.838Z)

```

719:     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
720:         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
721:         validation) is inside `result` ? job status 'done' means 'the worker finished'."""
722:         return self._execute_write(
723:             "UPDATE jobs SET status='done', progress='finished', result=?, "
724:             "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
725:             (json.dumps(result), job_id),
726:             f"Failed to mark job {job_id} done",
727:         )
728:
729:     def mark_job_failed(self, job_id: str, error: str) -> bool:

```

```

730-         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
731-     return self._execute_write(
732-         "UPDATE jobs SET status='failed', error=?, "
733-         "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id),
734-         f"Failed to mark job {job_id} failed",
735-     )
736-
737-     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
738-         """Fetch one job; `result`/`player_pool` JSON deserialized."""
739-         with self._connect() as conn:
740-             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
741-             if not row:
742-                 return None
743-             d = dict(row)
744-             d["result"] = json.loads(d["result"]) if d["result"] else None

```

23. assistant (2026-06-22T18:25:33.313Z)

Confirmed: a segmenter-raise ? worker `except` ? `mark\_job\_failed` stores only an error string, `result` stays NULL ? `/api/jobs/{id}` returns `result: None`. So the worker-crash path has no `compute` block at all, which is consistent (the change's docstring says "not_run" only applies to dict responses, and a raise means the response was never built). This is acceptable behavior ? not a bug per se, but the docstring claim that "for a segmenter that raises it DID start running in-process" is NOT reflected in any result (there's no result). The `compute\_actual="in\_process"` is set, but `response` is None, so it's never stamped. Let me verify this is what the task is probing.

Let me now check existing consumers of the job result / `compute` key ? SPA contract and any test that asserts exact result shape.

24. user (2026-06-22T18:25:34.611Z)

Found 10 files

```

C:\Users\avidu\Projects\bhi-computeval\scripts\verify_compute_path.py
C:\Users\avidu\Projects\bhi-computeval\tests\test_api_cloud_dispatch.py
C:\Users\avidu\Projects\bhi-computeval\backend\main.py
C:\Users\avidu\Projects\bhi-computeval\backend\compute\cloud_run.py
C:\Users\avidu\Projects\bhi-computeval\tests\test_database.py
C:\Users\avidu\Projects\bhi-computeval\tests\test_compute_backend.py
C:\Users\avidu\Projects\bhi-computeval\tests\test_async_jobs.py
C:\Users\avidu\Projects\bhi-computeval\backend\worker\__main__.py
C:\Users\avidu\Projects\bhi-computeval\backend\infrastructure\database.py
C:\Users\avidu\Projects\bhi-computeval\backend\compute\__init__.py

```

25. assistant (2026-06-22T18:25:37.858Z)

Let me check the tests that assert exact response shape ? specifically the ones that might break on the new always-present `compute` key. Let me look for tests asserting the in-process success/fail response shape with exact dict equality.

26. user (2026-06-22T18:25:39.850Z)

```

C:\Users\avidu\Projects\bhi-computeval\tests\test_ablation.py:139:    assert
set(shuttle.columns) == {"duration", "peak_velocity", "mean_velocity", "active_ratio"}
C:\Users\avidu\Projects\bhi-computeval\tests\test_ablation.py:153:    assert set(rows) ==
{"shuttle", "gate_a", "person", "audio"}
C:\Users\avidu\Projects\bhi-computeval\tests\test_ablation.py:383:    assert {v.name for v in
videos} == {f"v{k}" for k in range(6)}
C:\Users\avidu\Projects\bhi-computeval\tests\test_ablation.py:509:    assert {r["group"] for r
in report["rows"]} == {"shuttle", "gate_a", "person", "audio"}
C:\Users\avidu\Projects\bhi-computeval\tests\test_api_cloud_dispatch.py:119:    assert
{md.get("shot_count") for md in metas} == {5, 3}
C:\Users\avidu\Projects\bhi-computeval\tests\test_api_cloud_dispatch.py:120:    assert

```

```

{md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}
C:\Users\avidu\Projects\bhi-computeval\tests\test_api_cloud_dispatch.py:448:    assert
out["compute"] == {"path": "in_process", "requested": "local"}
C:\Users\avidu\Projects\bhi-computeval\tests\test_api_cloud_dispatch.py:463:    assert
out["compute"] == {"path": "in_process", "requested": None}
C:\Users\avidu\Projects\bhi-computeval\tests\test_api_cloud_dispatch.py:481:    assert
out["compute"] == {"path": "cloud_run", "requested": "cloud", "dispatched": False}
C:\Users\avidu\Projects\bhi-computeval\tests\test_api_endpoints.py:137:    assert names ==
{"reel_a.mp4", "reel_b.mov"} # only top-level video files
C:\Users\avidu\Projects\bhi-computeval\tests\test_api_endpoints.py:176:    assert
db.count_jobs_by_locale() == {"kn": 1}
C:\Users\avidu\Projects\bhi-computeval\tests\test_automated_provider.py:32:    assert out ==
[(0.0, 10.0), (20.0, 30.0)] # sorted by start
C:\Users\avidu\Projects\bhi-computeval\tests\test_audio_features.py:89:    assert {v.name for v
in videos} == {"fusion0", "fusion1"} # windowing-only files skipped
C:\Users\avidu\Projects\bhi-computeval\tests\test_billing.py:17:    assert cost == 0.0 and bd ==
{"gpu_seconds": 0.0, "egress_gb": 0.0}
C:\Users\avidu\Projects\bhi-computeval\tests\test_billing.py:77:    assert
db.get_pricebook_rates("nonexistent", "L4") == {} # unknown version ? no rates ? cost 0
C:\Users\avidu\Projects\bhi-computeval\tests\test_cleanup_caches.py:43:    assert names ==
{"v_frames", "v.mp4"}
C:\Users\avidu\Projects\bhi-computeval\tests\test_cleanup_caches.py:44:    assert {e.name for e
in kept} == {"v_traj.csv", "v_traj_wasbcache"}
C:\Users\avidu\Projects\bhi-computeval\tests\test_cleanup_caches.py:101:    assert {e.name for e
in to_del} == {"stale.mp4", ".partial-abc"}
C:\Users\avidu\Projects\bhi-computeval\tests\test_cleanup_caches.py:102:    assert {e.name for e
in kept} == {"fresh.mp4", ".partial-live"}
C:\Users\avidu\Projects\bhi-computeval\tests\test_cleanup_caches.py:147:    assert {e.name for e
in win} == {"GX_frames"} # uploads dir not double-listed
C:\Users\avidu\Projects\bhi-computeval\tests\test_cleanup_caches.py:148:    assert {e.name for e
in ups} == {"match.mp4", ".partial-z"}
C:\Users\avidu\Projects\bhi-computeval\tests\test_config.py:68:    assert f.cue_flags == {}
# person cue OFF
C:\Users\avidu\Projects\bhi-computeval\tests\test_config.py:108:    assert f.cue_flags ==
{"person": True}
C:\Users\avidu\Projects\bhi-computeval\tests\test_compute_backend.py:150:    assert data ==
{"video_id": "v", "returncode": 0}
C:\Users\avidu\Projects\bhi-computeval\tests\test_config_deployment.py:68:    assert
dumped["deployment"] == {"profile": "", "compute_target": ""}
C:\Users\avidu\Projects\bhi-computeval\tests\test_config_deployment.py:78:    assert
literal_profiles == {"", *PROFILES}
C:\Users\avidu\Projects\bhi-computeval\tests\test_config_deployment.py:87:    assert
literal_targets == {"", *COMPUTE_TARGETS}
C:\Users\avidu\Projects\bhi-computeval\tests\test_config_deployment.py:351:    assert
preset_for("") == {}
C:\Users\avidu\Projects\bhi-computeval\tests\test_config_deployment.py:352:    assert
preset_for("banana") == {}
C:\Users\avidu\Projects\bhi-computeval\tests\test_config_deployment.py:358:    assert
saved["deployment"] == {"profile": "", "compute_target": ""}
C:\Users\avidu\Projects\bhi-computeval\tests\test_database.py:312:    assert cols == {
C:\Users\avidu\Projects\bhi-computeval\tests\test_database.py:337:    assert m["fusion_config"]
== {"min_crossings": 3, "cue_flags": {"person": True}}
C:\Users\avidu\Projects\bhi-computeval\tests\test_database.py:364:    assert m["fusion_config"]
== {"min_crossings": 5}
C:\Users\avidu\Projects\bhi-computeval\tests\test_database.py:397:    assert m["fusion_config"]
== {"min_crossings": 2}
C:\Users\avidu\Projects\bhi-computeval\tests\test_database.py:410:    assert m["fusion_config"]
== {"min_crossings": 4}
C:\Users\avidu\Projects\bhi-computeval\tests\test_eval_regression.py:33:    assert
regression.load_baselines(str(tmp_path / "nope.json")) == {}
C:\Users\avidu\Projects\bhi-computeval\tests\test_export_wasb_segments.py:49:    assert
load_rally_labels(out) == {"1": (1.0, 3.0), "2": (5.5, 8.25)}
C:\Users\avidu\Projects\bhi-computeval\tests\test_fusion_compare.py:55:    assert {v.name for v
in videos} == {"fusion0", "fusion1"} # windowing-only files skipped
C:\Users\avidu\Projects\bhi-computeval\tests\test_fusion_golden_telemetry.py:88:    assert

```

```
{r["msg"] for r in starts} == {"alpha", "beta"}
C:\Users\avidu\Projects\bhi-computeval\tests\test_flywheel.py:108:    assert res == {"ok": 1}
```

[Showing results with pagination = limit: 40]

27. assistant (2026-06-22T18:25:43.747Z)

No existing test asserts an exact full-dict equality on the `/api/process` job result that would now break on the new key (the matches are all on unrelated dicts; the `compute` assertions are the new tests). Let me confirm no test does a strict `.keys()` equality on the process response, and check the SPA contract (TypeScript/JS) to see if any client validates exact result keys.

28. user (2026-06-22T18:25:45.479Z)

Found 2 files

```
C:\Users\avidu\Projects\bhi-computeval\backend\ui\assets\index-ukuS9wxa.js
C:\Users\avidu\Projects\bhi-computeval\backend\api\static\app.js
```

29. assistant (2026-06-22T18:25:56.101Z)

These are bundled SPA assets (the engine serves the SPA, the source lives in the sibling khelsutra repo). A new additive key in the JSON result won't break a JS consumer that reads specific fields. The `compute` key is purely additive and read by-name, so existing consumers are unaffected.

Now let me run the new tests plus the broader cloud-dispatch/async test suite to confirm green and check for regressions.

30. user (2026-06-22T18:26:10.243Z)

```
..... [100%]
58 passed in 10.36s
```

31. assistant (2026-06-22T18:26:18.533Z)

All 58 pass. Now let me check the `\_FakeCloudClient` to confirm the test's `execution == "exec-1"` assertion is grounded? and verify the empty-execution edge case (run_job returning None) doesn't crash. Let me look at the fake and the test helpers.

32. user (2026-06-22T18:26:20.448Z)

```
23:from backend.infrastructure.execution_policy import ExecutionPolicy
29:def _pass():
38:class _FakeCloudClient(CloudRunJobClient):
48:    def run_job(self, job_name, argv, *, region, project=None):
67:        return "exec-1"
71:def cloud_env(tmp_path, monkeypatch):
92:def _inject_fake(monkeypatch, fake):
106:def test_cloud_dispatch_ingests_segments(cloud_env):
107:    db, _bucket, monkeypatch = cloud_env
108:    fake = _FakeCloudClient()
109:    _inject_fake(monkeypatch, fake)
123:def test_cloud_dispatch_builds_correct_spec(cloud_env):
124:    _db, bucket, monkeypatch = cloud_env
125:    fake = _FakeCloudClient()
126:    _inject_fake(monkeypatch, fake)
130:    assert len(fake.calls) == 1 # exactly one Cloud Run Job execution
140:def test_cloud_failure_marks_failed_no_rows(cloud_env):
141:    db, _bucket, monkeypatch = cloud_env
144:    fake = _FakeCloudClient(returncode=1)
145:    _inject_fake(monkeypatch, fake)
154:def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
155:    _db, _bucket, monkeypatch = cloud_env
156:    fake = _FakeCloudClient()
```

```

157:     _inject_fake(monkeypatch, fake)
177:     class _InProcSeg:
185:         monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
192: def test_cloud_ingest_failure_marks_failed_no_leak(cloud_env):
195:     db, _bucket, monkeypatch = cloud_env
197:     class _NoCsvClient(_FakeCloudClient):
198:         def run_job(self, job_name, argv, *, region, project=None):
212:     _inject_fake(monkeypatch, _NoCsvClient())
219: def test_cloud_video_id_divergence_is_tolerated_with_warning(cloud_env, caplog):
222:     db, _bucket, monkeypatch = cloud_env
223:     _inject_fake(monkeypatch, _FakeCloudClient(video_id="DIVERGED"))
231: class _InProcSeg:
243: def test_per_request_local_forces_in_process(cloud_env):
246:     db, _bucket, monkeypatch = cloud_env
247:     fake = _FakeCloudClient()
248:     _inject_fake(monkeypatch, fake)
249:     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)

```

33. assistant (2026-06-22T18:26:26.881Z)

Now let me verify the empty-execution edge case concretely. The ``GoogleCloudRunJobClient.run_job`` (line 79) can return the literal string ``"execution"`` as fallback, never `None`, but ``run_infer`` defensively does ``str(execution or "")``. If a future/custom client returned ``None``, ``execution=""`` (empty string, no crash) and the provenance would have ``execution: ""``. The success message still makes sense, just unprovable. That's a graceful degradation, not a bug. Let me write a quick sanity check to confirm the empty-execution path and the ``_failed(dispatched=True)`` flag interpretation.

34. user (2026-06-22T18:26:29.144Z)

```

empty-exec ok, execution= ''
default execution= ''

```

35. assistant (2026-06-22T18:26:39.195Z)

Now let me probe the most important provenance-accuracy concern: the ``_failed`` path with ``dispatched=True``. When the cloud job genuinely fails (returncode != 0, dispatch error, or ingest error) the result is ``{path: cloud_run, requested: cloud, dispatched: True}`` but status ``failed``. That's accurate ? it dispatched and ran in cloud but failed. Good.

But there's a subtler case I want to verify: the ``finally`` ``setdefault`` interaction with the cloud ``_failed`` branch. When ``_maybe_dispatch_remote`` returns a ``_failed`` tuple, ``_execute_processing`` sets ``response, segmentation_ran = remote`` and returns at line 694. The ``finally`` then runs ``setdefault`` ? the cloud branch already set ``response["compute"]``, so ``setdefault`` is a no-op and the cloud value (with ``dispatched``) survives. Confirmed correct.

Let me double-check one provenance subtlety the task flagged: in the cloud ``_failed(dispatched=False)`` case, ``compute.path = "cloud_run"``. Could a reader misread ``path: cloud_run`` + ``dispatched: False`` as "ran in cloud"? Let me check what the report harvest does with ``compute_path`` and whether the cloud ``_failed`` path even reaches the report seam with the right value.

In the cloud ``_failed`` path, ``_execute_processing`` returns ``response`` at line 694 with ``compute.path = "cloud_run"``, but note ``compute_actual`` is still ``None`` (it was never set to ``"in_process"`` because we returned before line 696). The ``finally`` computes ``compute_path = response["compute"]["path"]`` = ``"cloud_run"``. So the ``**report`` for a pre-dispatch-reject (``dispatched=False``) records ``compute_path="cloud_run"`` even though nothing ran remotely. Let me verify this is the actual behavior with a focused trace.

36. user (2026-06-22T18:26:46.984Z)

```

23:56:46 [INFO] backend.main: Running validations for
C:\Users\avidu\AppData\Local\Temp\tmpjbs36ve0\v.mp4...
23:56:46 [INFO] backend.main: [API] Video passed checks. Segmenting and indexing...

```

```
23:56:46 [INFO] backend.main: [API] Using segmenter: wasb_hybrid
23:56:46 [INFO] backend.main: [COMPUTE] video_id=1509ff120e05e908896a03d62cf7ef1c
requested=cloud actual=cloud_run status=failed
status= failed
result.compute= {'path': 'cloud_run', 'requested': 'cloud', 'dispatched': False}
report compute_path= cloud_run
```

37. assistant (2026-06-22T18:26:56.885Z)

This confirms the behavior the task flagged. Now let me weigh whether ``path: "cloud_run"` + `dispatched: False` is misleading. The `[COMPUTE]` log line says `actual=cloud_run status=failed` for a job that never ran on cloud (and never ran anywhere ? it was rejected pre-dispatch because the server isn't cloud-configured).`

This is the one genuinely arguable provenance concern. The result dict carries ``dispatched: False`` which disambiguates, but:

1. The ``[COMPUTE]`` log line emits ``actual=cloud_run`` with **no ``dispatched`` field** ? a log reader sees ``actual=cloud_run`` and would reasonably conclude the cloud path was taken, when in fact nothing ran.
2. The **report** seam receives ``compute_path="cloud_run"`` with no ``dispatched`` qualifier ? the obsreport record will say the compute path was ``cloud_run`` for a job that ran nowhere.

The docstring for ``compute_path`` in reporting explicitly says the values are ``in_process / cloud_run / not_run``. A pre-dispatch reject is conceptually ``not_run`` (nothing executed), but it's recorded as ``cloud_run``. This is an inconsistency between the result dict (which has the ``dispatched: False`` escape hatch) and the report/log (which don't). Let me confirm the report's ``not_run`` semantics are documented this way and check whether this is a real "misleading provenance" finding vs. acceptable.

Let me verify the in-process success path produces a clean, byte-identical-plus-one-key result and run the full relevant test set once more including the report path with obsreport. Let me check if obsreport is installed (so the report seam is actually exercised).

38. user (2026-06-22T18:26:59.063Z)

```
obsreport AVAILABLE C:\Users\avidu\Projects\sports-obsreport\src\obsreport\__init__.py
---
OBSREPORT_AVAILABLE= True
```

39. assistant (2026-06-22T18:27:03.776Z)

obsreport is installed, so the report seam is live. Let me verify ``harvest.record_run`` accepts ``run_signals`` with the new ``compute_path`` key without error (i.e., obsreport doesn't validate/reject unknown signal keys). The earlier full test run already exercised ``emit_indexer_report`` with obsreport available and passed, but let me confirm the harvest passes ``run_signals`` through generically.

40. user (2026-06-22T18:27:04.551Z)

```
139:         run_signals: Optional[Dict[str, Any]] = None) -> None:
142:     run_signals (backend.reporting.run_signals), e.g. oversize chunk refusals."""
156:         "oversize_clips_skipped": int((run_signals or {}).get("oversize_clips_skipped",
0)) if ai_based else None,
177: def record_run(
192:     run_signals: Optional[Dict[str, Any]] = None,
219:     _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran, run_signals)
```

41. user (2026-06-22T18:27:07.512Z)

```
219         _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran,
run_signals)
220         _highlights(rec, play_segments, vm)
221
222         rec.finalize()
```

```
223         return rec
224
```

42. assistant (2026-06-22T18:27:13.408Z)

So `run\_signals` is consumed selectively by `harvest` (only `oversize\_clips\_skipped` at line 156); the new `compute\_path` key in `signals` is ****silently ignored**** by harvest ? it does NOT actually flow into the report. Let me verify this is the case: the diff comment at reporting/___init__.py line 58 says `# flows into the report via record\_run(run\_signals=?)` , but harvest only reads `oversize\_clips\_skipped` from `run\_signals`. The `compute\_path` is passed in the dict but never read/recorded by harvest. Let me confirm harvest doesn't generically dump all run_signals into the record.

43. user (2026-06-22T18:27:14.119Z)

```
130         if ran and segmenter_actual:
131             st.details["metadata_source"] = (
132                 "synthetic/heuristic ? NOT measured" if segmenter_actual == "motion"
133                 else "measured (detector/AI)"
134             )
135
136
137     def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:
Optional[str],
138                 play_segments: List[Dict[str, Any]], ran: bool,
139                 run_signals: Optional[Dict[str, Any]] = None) -> None:
140         """Only meaningful for AI-based segmenters. Records whether the provider key was
141         present ? the signal the silent_provider_noop rule keys on ? plus any in-process
142         run signals (backend.reporting.run_signals), e.g. oversize chunk refusals."""
143         ai_based = segmenter_actual in _AI_SEGMENTERS
144         provider = (config.get("ai_provider") or "gemini")
145         key_env = f"{provider.upper()}_API_KEY"
146         with rec.stage("ai_handoff") as st:
147             if not ran or not ai_based:
148                 st.status = "not_run" if not ran else "skipped"
149             st.details.update({
150                 "ai_based": ai_based,
151                 "provider": provider if ai_based else None,
152                 "model": config.get("ai_model") if ai_based else None,
153                 "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
154                 # Chunks the provider refused as over its upload cap (no analysis ran for
155                 # those time ranges). Detail name couples to the oversize_clip_skipped rule.
156                 "oversize_clips_skipped": int((run_signals or
157 {})).get("oversize_clips_skipped", 0)) if ai_based else None,
158             })
159             if ai_based:
160                 st.add_metric("confirmed_segments", len(play_segments))
161
162
163     def _highlights(rec: "obsreport.RunRecorder", play_segments: List[Dict[str, Any]],
164                   video_meta: Dict[str, Any]) -> None:
165         total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
166         coverage = None
167         dur = video_meta.get("duration_s")
168         if dur:
169             coverage = round(100.0 * total / dur, 1)
170         notes = []
171         if not play_segments:
172             notes.append("No highlights were produced for this video.")
173         rec.set_highlights(segment_count=len(play_segments), total_highlight_s=total,
174                           coverage_pct=coverage, notes=notes)
175
176     # --- top-level -----
177     def record_run(
```

```
178         *,
179         video_id: str,
```

44. assistant (2026-06-22T18:27:20.286Z)

This confirms a real finding. `harvest.record\_run` / `\_ai\_stage` only ever reads `oversize\_clips\_skipped` from `run\_signals`. The `compute\_path` value placed into `signals` by `emit\_indexer\_report` (reporting/`\_\_init\_\_.py`:57-58) is **never** read by harvest and never written into the report record². So the comment `# flows into the report via record\_run(run\_signals=?)` and the function docstring's claim ("so a run's report proves the compute path") are **false** ? the compute path does NOT appear in the obsreport output at all. It only appears in the `[COMPUTE]` log line and the `result["compute"]` dict.

Let me confirm by checking the actual written report JSON for the compute path. Let me grep the harvest module and obsreport for any generic dump of signals.

45. user (2026-06-22T18:27:24.873Z)

```
backend/reporting/harvest.py:139:         run_signals: Optional[Dict[str, Any]] = None) ->
None:
backend/reporting/harvest.py:142:     run signals (backend.reporting.run_signals), e.g. oversize
chunk refusals. """
backend/reporting/harvest.py:156:         "oversize_clips_skipped": int((run_signals or
{}).get("oversize_clips_skipped", 0)) if ai_based else None,
backend/reporting/harvest.py:192:     run_signals: Optional[Dict[str, Any]] = None,
backend/reporting/harvest.py:219:     _ai_stage(rec, config, segmenter_actual, play_segments,
segmentation_ran, run_signals)
backend/reporting/spec.yaml:45: # observed live 2026-06-10. The count flows segmenter ->
run_signals -> ai_handoff.
backend/reporting/__init__.py:12:from . import run_signals # no obsreport dependency ? always
importable
backend/reporting/__init__.py:39:     compute_path: Optional[str] = None,
backend/reporting/__init__.py:44:     ``compute_path`` (the validation method) records which
backend ran the DETECT ?
backend/reporting/__init__.py:56:     signals = run_signals.pop(video_id)
backend/reporting/__init__.py:57:     if compute_path is not None:
backend/reporting/__init__.py:58:         signals["compute_path"] = compute_path # flows into
the report via record_run(run_signals=?)
backend/reporting/__init__.py:76:         run_signals=signals,
Binary file backend/reporting/__pycache__/harvest.cpython-313.pyc matches
Binary file backend/reporting/__pycache__/run_signals.cpython-313.pyc matches
Binary file backend/reporting/__pycache__/__init__.cpython-313.pyc matches
=== obsreport record search ===
```

46. assistant (2026-06-22T18:27:30.070Z)

Confirmed. `compute\_path` is set into `signals` but **no** consumer reads it² ? not `harvest.record\_run`, not `\_ai\_stage`, not obsreport. The report never records the compute path. This is a real (minor) finding: the change's stated goal ("a run's report proves the compute path") is not met for the obsreport output, and the inline comment is misleading. The proof IS available in `result["compute"]` (the job result) and the `[COMPUTE]` log, so the core provenance feature works ? it's just the report-seam wiring and its docstring/comment that overclaim.

Let me do a final empirical confirmation: write an actual report and grep the output for the compute path, to be 100% sure it's absent.

47. user (2026-06-22T18:27:35.917Z)

```
paths: {'json': 'C:\\Users\\avidu\\AppData\\Local\\Temp\\tmplbh7c85z\\x.report.json',
'technical_md': 'C:\\Users\\avidu\\AppData\\Local\\Temp\\tmplbh7c85z\\x.report.md',
'customer_md': 'C:\\Users\\avidu\\AppData\\Local\\Temp\\tmplbh7c85z\\x.summary.md'}
C:\\Users\\avidu\\AppData\\Local\\Temp\\tmplbh7c85z\\x.report.json -> contains compute_path? False |
contains in_process? False
```

```
C:\Users\avidu\AppData\Local\Temp\tmplbh7c85z\x.report.md -> contains compute_path? False |
contains in_process? False
C:\Users\avidu\AppData\Local\Temp\tmplbh7c85z\x.summary.md -> contains compute_path? False |
contains in_process? False
```

48. assistant (2026-06-22T18:27:42.438Z)

Empirically confirmed: `compute\_path` does **not** appear anywhere in the emitted report (JSON, technical MD, or customer MD). The report seam is wired (`signals["compute\_path"]` is set) but there's no consumer, so the report does not prove the compute path ? contradicting both the inline comment (`# flows into the report`) and the `emit\_indexer\_report` docstring (`so a run's report proves the compute path`).

Note this is safe (it can't raise/break a run ? the unread key is harmless), so it's a documentation-accuracy / dead-wiring finding, not a correctness break.

Let me do a final lint/type check on the touched files to round out the review.

49. user (2026-06-22T18:27:47.244Z)

All checks passed!

```
=== mypy ===
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\compute\cloud_run.py:74: error: Argument 1 to "job_path" of "JobsClient" has
incompatible type "str | None"; expected "str" [arg-type]
Found 1 error in 1 file (checked 2 source files)
```

50. assistant (2026-06-22T18:28:25.566Z)

The one mypy error (`cloud\_run.py:74`, `job\_path` with `str | None` project) is **pre-existing** ? it's in `GoogleCloudRunJobClient.run\_job`, untouched by this diff (the diff only modified line 132-133, the `ComputeResult` construction). Let me confirm it's pre-existing by checking the diff doesn't touch line 74 (it doesn't ? verified above; the only cloud_run.py change is the `execution=` arg on the return). So it's not introduced by this change.

Lint is clean. Now I have everything I need. Let me compile the findings.

Adversarial review ? compute-path provenance

I traced every return path of `execute\_processing`, the `finally`/`setdefault` interplay, `\_maybe\_dispatch\_remote`, the `execution` threading, the report seam, and the verify script; ran the relevant suites (58 passed), and lint is clean.

Findings

****1. MINOR ? Misleading claim + dead wiring: `compute\_path` never reaches the report.****

```
`backend/reporting/__init__.py:57-58`
```python
if compute_path is not None:
 signals["compute_path"] = compute_path # flows into the report via
record_run(run_signals=?)
```
```

The comment and the `emit\_indexer\_report` docstring (`backend/reporting/\_\_init\_\_.py:44-45`, "so a run's report proves the compute path") both claim the compute path flows into the report. It does not. `harvest.record\_run` ? `\_ai\_stage` (`backend/reporting/harvest.py:139,156`) reads exactly one key from `run\_signals` ? `oversize\_clips\_skipped` ? and ignores everything else; there is no generic dump of `run\_signals` into the record. I emitted a real report with `compute\_path="in\_process"` and confirmed the string appears in **none** of the three outputs (JSON / technical MD / customer MD).

Why it's real: a stated goal of the change ("a run's report proves the compute path") is unmet, and the inline comment asserts a data flow that doesn't exist ? a future reader will trust the report to carry provenance and it won't.

Not a correctness/crash risk: the unread key is harmless (the seam never raises; ``run_signals.pop`` always returns a dict). The provenance IS genuinely available in ``result["compute"]`` and the ``[COMPUTE]`` log ? only the `*report*` claim is false.

Fix: either (a) record it in obsreport ? add a ``compute_path`` detail to a stage in ``_ai_stage`/`record_run`` (it's currently AI-segmenter-gated, so it would need its own non-AI-gated detail), or (b) drop the false claim: remove the "flows into the report" comment and the docstring's "so a run's report proves the compute path", keeping the result-dict + log as the proof surface.

****2. MINOR (judgment call) ? ``path: "cloud_run"`` recorded for a job that ran nowhere.****
``backend/main.py:525-527`` (``_failed``) and the resulting log at ``backend/main.py:742-745``.
For ``compute="cloud"`` on an unconfigured server (pre-dispatch reject), the result is ``{path: "cloud_run", requested: "cloud", dispatched: False}`` and the ``[COMPUTE]`` log emits ``actual=cloud_run status=failed``. I traced it live and confirmed. The result dict has the ``dispatched: False`` escape hatch so a careful reader can disambiguate ? but the ****log line has no ``dispatched`` field****, so ``actual=cloud_run`` there reads as "the cloud path was taken" when in fact nothing executed (conceptually this is the ``not_run`` case the design defines).
Why it's arguably real: the design's own vocabulary is ``in_process`` / ``cloud_run`` / ``not_run``, and a never-dispatched job is ``not_run``, not ``cloud_run``. The result dict is defensible (intent-vs-outcome with ``dispatched``), but the log is genuinely ambiguous.
Fix (optional): add ``dispatched`/outcome` to the ``[COMPUTE]`` log line, or treat ``dispatched: False`` as ``path: "not_run"`` (with ``requested: "cloud"``) so "what ran" and "what was asked" stay cleanly separated. Low severity ? the result dict already carries enough to disambiguate for the SPA/operator.

Everything else probed ? verdicts (no bug)

- ****Provenance accuracy across all 6 return paths:**** correct. validation-fail ? ``not_run``; segmenter-not-found ? ``not_run``; in-process success ? ``in_process``; in-process Failure ? ``in_process`` (``compute_actual`` set at ``main.py:696`` before the segmenter runs, survives into ``finally``); cloud success ? ``cloud_run`` + real execution; cloud ``_failed`` ? ``cloud_run`` + ``dispatched``. The ``finally`` ``setdefault`` correctly no-ops when the cloud branch already stamped ``response["compute"]``, and correctly stamps the others. No path can claim ``cloud_run`` when it ran in-process or vice-versa.
- ****Segmenter-raise path:**** the inner ``except`` (``main.py:700-705``) re-raises with ``response`` still ``None``; ``finally`` runs but ``isinstance(response, dict)`` is False, so nothing is stamped, and the worker (``job_worker.py:110-112``) marks the job ``failed`` with only an error string ? ``result`` is NULL, so ``/api/jobs/{id}`` returns ``result: None`` with ****no ``compute`` block****. Consistent (no false claim); the docstring's "it DID start running in-process" is simply not surfaced anywhere, which is acceptable. The ``finally`` IS reached on the raise (verified by control-flow).
- ****Default-OFF / new key / byte-identical:**** the in-process result is identical except for the additive ``compute`` key. No existing test asserts ``exact-dict/.keys()`` equality on the process result; the SPA consumers (bundled JS) read fields by name. No consumer breaks.
- ****`execution` threading / empty execution:**** ``run_infer`` does ``str(execution or "")`` ? empty string, never a crash, if a client returns None/empty. Real ``GoogleCloudRunJobClient.run_job`` falls back to the literal ``"execution"``, never None. Success provenance with an empty execution is still coherent (just unprovable). Fine.
- ****Report seam can't break a run:**** ``signals["compute_path"]`` assignment is safe (``pop`` always returns a dict); the only real-work block is inside ``try/except Exception: return None``; ``emit_indexer_report`` never raises. Confirmed.
- ****verify script:**** exit codes are correct ? 2 submit-fail, 3 poll-timeout, 4 path-mismatch, 5 missing-execution, 6 GCP-unconfirmed, 0 success. Poll loop terminates on ``done|failed`` with a monotonic timeout. ``--verify-gcp`` correctly fails non-zero when ``gcloud ? describe`` returns non-zero. No bug.

Local verification

- ``pytest tests/test_api_cloud_dispatch.py tests/test_async_jobs.py tests/test_compute_backend.py tests/test_reporting.py`` ? ****58 passed**** (obsreport IS installed, so the report seam ran live).
- ``ruff check`` on all touched files ? clean.
- ``mypy``: the one error (``backend/compute/cloud_run.py:74``, ``job_path`` with ``str | None``)

project) is **pre-existing** in `GoogleCloudRunJobClient`` and untouched by this diff (the only `cloud_run.py` change is the `execution=`` arg at line 132-133).

Net: No blockers, no correctness breaks, no default-OFF/backward-compat regression, no broken consumer. One real but minor accuracy issue (Finding 1: the "report proves the compute path" claim is false ? the value is set into `run_signals`` but no consumer reads it), plus one low-severity log/semantics ambiguity (Finding 2). The core feature ? provable provenance in `result["compute"]`` + the `[COMPUTE]`` log + GCP execution cross-check ? is correct.